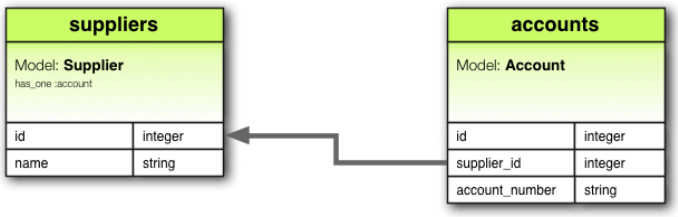
2.2 The has_one Association

A has_one association also sets up a one-to-one connection with another model, but with somewhat different semantics (and consequences). This association indicates that each instance of a model contains or possesses one instance of another model. For example, if each supplier in your application has only one account, you'd declare the supplier model like this:

```
class Supplier < ActiveRecord::Base
  has_one :account
end
```




```
class Supplier < ActiveRecord::Base
  has_one :account
end
```

The corresponding migration might look like this:

```
class CreateSuppliers < ActiveRecord::Migration
  def change
    create_table :suppliers do |t|
      t.string :name
      t.timestamps null: false
    end

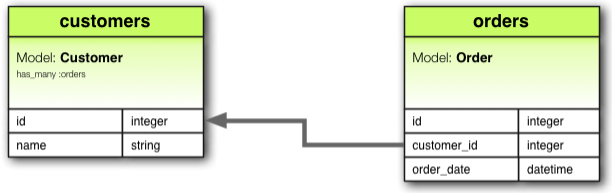
    create_table :accounts do |t|
      t.belongs_to :supplier, index: true
      t.string :account_number
      t.timestamps null: false
    end
  end
end
```

2.3 The has_many Association

A has_many association indicates a one-to-many connection with another model. You'll often find this association on the "other side" of a belongs_to association. This association indicates that each instance of the model has zero or more instances of another model. For example, in an application containing customers and orders, the customer model could be declared like this:

```
class Customer < ActiveRecord::Base
  has_many :orders
end
```

The name of the other model is pluralized when declaring a has_many association.




```
class Customer < ActiveRecord::Base
  has_many :orders
end
```